local-ai.js

Verified source file: renderer/local-ai.js

```
/*
 * FuturePowerAIPlus - Dual Local AI Engine
 *
 * Model A: LFM2.5-350M
 *   Fast everyday chat, short explanations and simple tasks.
 *
 * Model B: Qwen2.5-Coder-3B-Instruct
 *   Coding, harder mathematics, technical questions and longer answers.
 *
 * Runtime:
 *   1. Try WebGPU first.
 *   2. If WebGPU is unavailable/fails, try WASM/CPU.
 *   3. Never crash the webpage.
 *   4. Expose status through window.localAIStatus().
 *
 * Models are downloaded and cached locally by Transformers.js.
 */
import { pipeline, env } from "@huggingface/transformers";
/* ------------------------------------------------------------
   MODEL CONFIGURATION
------------------------------------------------------------ */
const FAST_MODEL = "onnx-community/LFM2.5-350M-ONNX";
const STRONG_MODEL = "onnx-community/Qwen2.5-Coder-3B-Instruct";
const FAST_MAX_TOKENS = 384;
const STRONG_MAX_TOKENS = 768;
/* ------------------------------------------------------------
   TRANSFORMERS.JS CONFIGURATION
------------------------------------------------------------ */
env.useBrowserCache = true;
env.allowRemoteModels = true;
env.allowLocalModels = false;
try {
  if (env.backends && env.backends.onnx && env.backends.onnx.wasm) {
    env.backends.onnx.wasm.numThreads = Math.max(
      1,
      Math.min(4, navigator.hardwareConcurrency || 2)
    );
    env.backends.onnx.wasm.simd = true;
  }
} catch (error) {
  console.warn("[FuturePowerAIPlus] WASM configuration warning:", error);
}
/* ------------------------------------------------------------
   STATE
------------------------------------------------------------ */
let fastPipe = null;
let strongPipe = null;
let fastLoading = null;
let strongLoading = null;
let fastDevice = null;
let strongDevice = null;
let lastError = null;
/* ------------------------------------------------------------
   STATUS SYSTEM
------------------------------------------------------------ */
function emitStatus(model, message, state = "loading", extra = {}) {
  const detail = {
    model,
    message,
    state,
    ...extra
  };
  try {
    window.dispatchEvent(
      new CustomEvent("futurepower-local-ai-status", { detail })
    );
  } catch (error) {
    console.warn("[FuturePowerAIPlus] Status event failed:", error);
  }
  console.info(`[FuturePowerAIPlus ${model}] ${message}`);
}
/* ------------------------------------------------------------
   BASIC ENVIRONMENT DETECTION
------------------------------------------------------------ */
function hasWebGPU() {
  return typeof navigator !== "undefined" && !!navigator.gpu;
}
function getEnvironment() {
  return {
    webGPU: hasWebGPU(),
    hardwareConcurrency:
      typeof navigator !== "undefined"
        ? navigator.hardwareConcurrency || 0
        : 0,
    userAgent:
      typeof navigator !== "undefined"
        ? navigator.userAgent
        : "unknown"
  };
}
window.localAIEnvironment = getEnvironment;
/* ------------------------------------------------------------
   DEVICE SELECTION
------------------------------------------------------------ */
async function selectDevice(model) {
  if (hasWebGPU()) {
    try {
      emitStatus(
```

```
        model,
        "WebGPU detected. Initializing GPU inference...",
        "loading"
      );
      const adapter = await navigator.gpu.requestAdapter();
      if (adapter) {
        emitStatus(model, "WebGPU adapter available.", "loading");
        return "webgpu";
      }
      emitStatus(
        model,
        "WebGPU exists but no GPU adapter was available. Trying CPU.",
        "warning"
      );
    } catch (error) {
      console.warn(
        "[FuturePowerAIPlus] WebGPU initialization failed:",
        error
      );
      emitStatus(
        model,
        "WebGPU initialization failed. Trying CPU/WASM.",
        "warning",
        { error: String(error?.message || error) }
      );
    }
  } else {
    emitStatus(
      model,
      "WebGPU is unavailable. Using CPU/WASM.",
      "warning"
    );
  }
  return "wasm";
}
/* ---------------------------------------------------------
   TASK ROUTER
--------------------------------------------------------- */
function taskFor(message) {
  const text = String(message || "").trim().toLowerCase();
  if (
    /\b(code|coding|program|programming|javascript|typescript|python|java|c\+\+|c#|html|css|sql|api|debug|debugging|bug|error|function|class|algorithm|re
gex|script|developer|software|github|json|node|npm|electron|react|webgpu)\b/.test(text)
  ) {
    return "strong";
  }
  if (
    /\b(solve|derive|prove|equation|quadratic|calculus|integral|integrate|derivative|differentiate|matrix|vector|probability|permutation|combination|trig
onometry|geometry|algebra|numerical|calculate|factorize|factorise|simplify)\b/.test(text)
  ) {
    return "strong";
  }
  if (/[=^√∫Σπ%]/.test(text) && /\d/.test(text)) {
    return "strong";
  }
  if (
    text.length > 650 ||
    /\b(summarize|summarise|summary|essay|report|article|detailed|compare|comparison|rewrite|research|chapter|notes|explain in detail|detailed explanatio
n)\b/.test(text)
  ) {
    return "strong";
  }
  return "fast";
}
window.localAISelectModel = taskFor;
/* ---------------------------------------------------------
   SYSTEM PROMPTS
--------------------------------------------------------- */
function messagesFor(message, model) {
  const userMessage = String(message || "").trim();
  if (model === "strong") {
    return [
      {
        role: "system",
        content:
          "You are FuturePowerAIPlus Strong, a capable local AI assistant. " +
          "You are especially useful for mathematics, programming, debugging, " +
          "technical explanations, science numericals and detailed writing. " +
          "Answer directly. For mathematics, show useful calculation steps. " +
          "For programming, provide practical runnable code when appropriate. " +
          "Do not invent facts. If you are uncertain, say so."
      },
      { role: "user", content: userMessage }
    ];
  }
  return [
    {
      role: "system",
      content:
        "You are FuturePowerAIPlus Fast, a helpful local everyday AI assistant. " +
        "Answer questions clearly and accurately. Be concise but useful. " +
        "For simple explanations, use easy language and examples when helpful."
    },
    { role: "user", content: userMessage }
  ];
}
/* ---------------------------------------------------------
   PIPELINE CREATION
--------------------------------------------------------- */
async function createPipeline(modelName, modelKey) {
  const device = await selectDevice(modelKey);
  emitStatus(
    modelKey,
    device === "webgpu"
```

```
      ? "Starting WebGPU model..."
      : "Starting CPU/WASM model...",
    "loading",
    { device }
  );
  if (device === "webgpu") {
    try {
      const pipe = await pipeline(
        "text-generation",
        modelName,
        {
          dtype: "q4",
          device: "webgpu"
        }
      );
      emitStatus(
        modelKey,
        "Local model ready using WebGPU.",
        "ready",
        { device: "webgpu" }
      );
      return { pipe, device: "webgpu" };
    } catch (error) {
      console.error(
        `[FuturePowerAIPlus ${modelKey}] WebGPU model loading failed:`,
        error
      );
      emitStatus(
        modelKey,
        "WebGPU model loading failed. Retrying with CPU/WASM.",
        "warning",
        { error: String(error?.message || error) }
      );
    }
  }
  try {
    const pipe = await pipeline(
      "text-generation",
      modelName,
      { device: "wasm" }
    );
    emitStatus(
      modelKey,
      "Local model ready using CPU/WASM.",
      "ready",
      { device: "wasm" }
    );
    return { pipe, device: "wasm" };
  } catch (error) {
    console.error(
      `[FuturePowerAIPlus ${modelKey}] CPU/WASM model loading failed:`,
      error
    );
    emitStatus(
      modelKey,
      "CPU/WASM model loading failed.",
      "error",
      { error: String(error?.message || error) }
    );
    throw error;
  }
}
/* ------------------------------------------------------------
   FAST MODEL
------------------------------------------------------------ */
async function loadFast() {
  if (fastPipe) return fastPipe;
  if (fastLoading) return fastLoading;
  fastLoading = (async () => {
    emitStatus("fast", "Preparing LFM2.5-350M...", "loading");
    try {
      const result = await createPipeline(FAST_MODEL, "fast");
      fastPipe = result.pipe;
      fastDevice = result.device;
      emitStatus(
        "fast",
        `LFM2.5-350M ready on ${fastDevice}.`,
        "ready",
        { device: fastDevice }
      );
      return fastPipe;
    } catch (error) {
      fastPipe = null;
      lastError = error;
      emitStatus(
        "fast",
        "LFM2.5-350M could not be loaded.",
        "error",
        { error: String(error?.message || error) }
      );
      throw error;
    }
  })().finally(() => {
    fastLoading = null;
  });
  return fastLoading;
}
/* ------------------------------------------------------------
   STRONG MODEL
------------------------------------------------------------ */
async function loadStrong() {
  if (strongPipe) return strongPipe;
  if (strongLoading) return strongLoading;
  strongLoading = (async () => {
```

```
    emitStatus(
      "strong",
      "Preparing Qwen2.5-Coder-3B...",
      "loading"
    );
    try {
      const result = await createPipeline(STRONG_MODEL, "strong");
      strongPipe = result.pipe;
      strongDevice = result.device;
      emitStatus(
        "strong",
        `Qwen2.5-Coder-3B ready on ${strongDevice}.`,
        "ready",
        { device: strongDevice }
      );
      return strongPipe;
    } catch (error) {
      strongPipe = null;
      lastError = error;
      emitStatus(
        "strong",
        "Qwen2.5-Coder-3B could not be loaded.",
        "error",
        { error: String(error?.message || error) }
      );
      throw error;
    }
  })().finally(() => {
    strongLoading = null;
  });
  return strongLoading;
}
/* ------------------------------------------------------------
   GENERATION
------------------------------------------------------------ */
async function generate(pipe, message, model) {
  const maxTokens =
    model === "strong"
      ? STRONG_MAX_TOKENS
      : FAST_MAX_TOKENS;
  const output = await pipe(
    messagesFor(message, model),
    {
      max_new_tokens: maxTokens,
      do_sample: false,
      return_full_text: false
    }
  );
  const generated = output?.[0]?.generated_text;
  if (Array.isArray(generated)) {
    const last = generated[generated.length - 1];
    return String(
      last?.content ??
      last ??
      ""
    ).trim();
  }
  return String(generated ?? "").trim();
}
/* ------------------------------------------------------------
   PUBLIC STATUS API
------------------------------------------------------------ */
window.localAIStatus = function () {
  return {
    available: true,
    webGPU: hasWebGPU(),
    fastLoaded: Boolean(fastPipe),
    strongLoaded: Boolean(strongPipe),
    fastLoading: Boolean(fastLoading),
    strongLoading: Boolean(strongLoading),
    fastDevice,
    strongDevice,
    fastModel: FAST_MODEL,
    strongModel: STRONG_MODEL,
    lastError:
      lastError
        ? String(lastError?.message || lastError)
        : null
  };
};
/* ------------------------------------------------------------
   PRELOAD API
------------------------------------------------------------ */
window.preloadLocalAI = async function(model = "fast") {
  if (model === "strong") {
    return loadStrong();
  }
  return loadFast();
};
/* ------------------------------------------------------------
   MAIN LOCAL AI API
------------------------------------------------------------ */
window.localAskAI = async function(message) {
  const text = String(message || "").trim();
  if (!text) return "";
  const selected = taskFor(text);
  emitStatus(
    selected,
    selected === "strong"
      ? "Using Strong AI for this task."
      : "Using Fast AI for this task.",
    "routing"
```

```
  );
  try {
    const pipe =
      selected === "strong"
        ? await loadStrong()
        : await loadFast();
    const answer = await generate(
      pipe,
      text,
      selected
    );
    if (answer) {
      emitStatus(
        selected,
        "Local AI response generated.",
        "ready"
      );
      return answer;
    }
    throw new Error(
      "The local model returned an empty response."
    );
  } catch (error) {
    lastError = error;
    console.warn(
      `[FuturePowerAIPlus] ${selected} model failed:`,
      error
    );
    emitStatus(
      selected,
      `${selected === "strong" ? "Strong" : "Fast"} model failed.`,
      "error",
      { error: String(error?.message || error) }
    );
    if (selected === "strong") {
      try {
        emitStatus(
          "fast",
          "Trying the smaller Fast model instead...",
          "loading"
        );
        const fast = await loadFast();
        const answer = await generate(
          fast,
          text,
          "fast"
        );
        if (answer) {
          emitStatus(
            "fast",
            "Fast-model fallback response generated.",
            "ready"
          );
          return answer;
        }
      } catch (fallbackError) {
        lastError = fallbackError;
        console.warn(
          "[FuturePowerAIPlus] Fast fallback also failed:",
          fallbackError
        );
      }
    }
    throw error;
  }
};
/* ------------------------------------------------------------
   STARTUP DIAGNOSTICS
------------------------------------------------------------ */
try {
  const environment = getEnvironment();
  console.info(
    "[FuturePowerAIPlus] Local AI engine initialized."
  );
  console.info(
    "[FuturePowerAIPlus] Environment:",
    environment
  );
  console.info(
    "[FuturePowerAIPlus] Fast model:",
    FAST_MODEL
  );
  console.info(
    "[FuturePowerAIPlus] Strong model:",
    STRONG_MODEL
  );
  if (environment.webGPU) {
    emitStatus(
      "system",
      "Local AI engine ready. WebGPU detected.",
      "ready"
    );
  } else {
    emitStatus(
      "system",
      "Local AI engine ready. WebGPU unavailable; CPU/WASM will be used.",
      "warning"
    );
  }
} catch (error) {
  lastError = error;
  console.error(
    "[FuturePowerAIPlus] Local AI startup error:",
    error
```

```
  );
}
```